



How to fix a USB-ISP programmer and make it work as USBasp with Arduino IDE on Linux

Introduction

Although I do not usually program chips, because I prefer to use complete modules, mainly for utilizing their USB port as a user interface, I happened to work on a few projects with a bare ATtiny85, which requires a programmer. I managed with a custom programmer, built on an Arduino Nano loaded with the `ArduinoISP` sketch. But as you all know, you never program once. Actually, I estimate that for program beautifications, minor adjustments etc. I program at least 5 times, and this, after the program is fully debugged!!! That is a lot of work, even for a single project.

So, I was tired of having bare PCBs and wires all over the place, therefore I decided to order a USB programmer in a brightly colored metal case. The device I ordered was described as **“Aluminum shell USB ISP USBISP USBASP ASP Programmer for 51 ATMELE AVR WIN7 64 (Random color)”** shown below.



I assumed that it would be the [USBasp - USB programmer for Atmel AVR controllers](https://www.fischl.de/usbasp/) [https://www.fischl.de/usbasp/]. I made the mistake of not making any research before I ordered but at a price of less than 2 Euros, it seemed a low risk.

After the product arrived, I tested it, considering it would function as a USBasp, but it would not work. Its light remained red at all times, not showing any sign of activity. I am working on a Linux Mint desktop and `dmesg` reported the USB device as:

```
$ dmesg
.....
usb 3-1: new low-speed USB device number 3 using ohci-platform
usb 3-1: New USB device found, idVendor=03eb, idProduct=c8b4
usb 3-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
usb 3-1: Product: USBHID
usb 3-1: Manufacturer: zhifengsoft
```

Googling the name of the manufacturer **“zhifengsoft”** as well as **“usb-isp”**, a ton of information came through. It seems that the device is working along with the Windows program [ProgISP](https://www.electrodragon.com/w/8051) [https://www.electrodragon.com/w/8051] and is not recognized by `avrdude`, the program which uploads code to Arduino and other Atmel micro-controllers.

Luckily, I did not have to re-invent the wheel because there has been a lot of excellent work. I decided to publish this article because:

1. I did not find online explicit information regarding the version of the PCB inside the USB-ISP (v5.00)
2. I have encountered many problems in the process and I had to consult several online sources. So I decided to gather in one place the complete procedure.

In order to make the device work on Linux and communicate with the Arduino IDE via `avrdude`, the solution is to re-flash the firmware of the USB-ISP, in order to make it behave as a USBasp, which in turn is recognized by `avrdude`. Luckily, we can use the pins of the exposed connector, as well as make a wire bridge to accomplish this re-flashing.

Note: `avrdude` has also an option to support programmers of type `usbasp-clone`, but I did not test it. You can see a list of all supported programmers with the command:

```
$ avrdude -c ?
```

So, to have a programmer, you need another programmer to re-flash it with the proper firmware!

To avoid any misconceptions:

- **USB-ISP** is the programmer device with the nice colored aluminum casing,
- **USBasp** is the firmware created by Thomas Fischl
- **ArduinoISP** is the sketch in *File* → *Examples* that makes Arduino behave as an `stk500v1` compatible programmer

The target audience for this article is mainly hobbyists. My opinion is that this device is by no means suitable for any corporate environment for production usage.

References:

1. Making USBasp Chinese Clones Usable by darell tan [http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/]
2. Hacking an AVR programmer - GreenPhotons the blog of uwezi [https://www.sciencetronics.com/greenphotons/?p=938]
3. Hacking an AVR programmer - GreenPhotons the blog of uwezi [https://www.sciencetronics.com/greenphotons/?p=1937]
4. USBasp - USB programmer for Atmel AVR controllers - Thomas Fischl [https://www.fischl.de/usbasp/]
5. USBasp on a clone - Aleh Dzenisiuk [https://github.com/aleh/usbisp]
6. `avrdude` Options Descriptions [http://www.nongnu.org/avrdude/user-manual/avrdude_4.html#Option-Descriptions]
7. Getting USBasp v2.0 to work with Linux Mint - Kevin's Blog [https://klamfx.wordpress.com/2017/05/18/getting-usbasp-v2-0-to-work-on-linux/]

Prerequisites:

The controller inside my USB-ISP is an **ATmega88**. As found online, other models have an **ATmega8**. Just remember it when you construct the parameters at the commands below.

You will need:

1. A programmer able to flash the controller of USB-ISP. I used an Arduino Nano running “`ArduinoISP`” from the IDE examples.
2. A 10-wire ribbon cable with female connectors on each side. In my case, it came with the USB-ISP.
3. Dupont wires (female on the Arduino Nano side - male on the ribbon cable side. For an Uno, male on the Arduino Uno side - male on the ribbon cable side)
4. A passive programming board. This is normally a ZIF socket or a breadboard which accommodates the chip that we want to program with our programmer and is wired to SCK, MISO, MOSI, RESET, as well as GND and Vcc. See also <https://www.arduino.cc/en/Tutorial/ArduinoToBreadboard> [https://www.arduino.cc/en/Tutorial/ArduinoToBreadboard] for typical wirings. This board is not mandatory to flash the USB-ISP but we would like to test that the entire programming process works end-to-end.
5. A spare MCU, to test our new programmer. My passive programming board is not equipped with a crystal, because I work a lot with the minimal configuration and without bootloaders, so the chip must be running under its own RC internal clock. Your configuration may be a board with a crystal, or do In-system programming at a board having a crystal

etc. There are hundreds of variations that I hope you are familiar with their differences.

WARNING

If you brick an MCU because you selected an improper clock source, there are solutions to revive your chip, but this is another story. Just keep in mind that the MCU which we are programming **needs its own working clock source at all times**. The signal SCK is only used for timing the SPI interface and **it is not a substitute for the MCU clock source**.

Proceed at your own risk.

Be cautious regarding the licensing of the USBasp firmware and especially the USB functionality which is implemented by V-USB [<https://www.obdev.at/products/vusb/license.html>].

Step 1. Identify your programming environment

Locate the path of avrdude. We will be using it a lot below.

Try:

```
$ sudo find / -name avrdude -print
$ sudo find / -name avrdude.conf -print
```

The executable is at somefolder/bin/avrdude and its configuration file is at somefolder/etc/avrdude.conf where somefolder is a path decided by your installation script or yourself. You may find several instances of the program and the configuration file. If, like me, you have several version of the Arduino IDE, each version has its own copy. In a multi-user environment, you may find also a hidden folder in the user's home directory, such as:

```
.arduino15/packages/arduino/tools/avrdude/6.3.0-arduino17/bin
.arduino15/packages/arduino/tools/avrdude/6.3.0-arduino17/etc/avrdude.conf
```

From all these copies, we only care about the one that is used by our Arduino IDE. To identify it, start the Arduino IDE. Select *File -> Preferences* and in the option *Show verbose output during*: check the *upload* check-box.

Load the **Blink** example and upload it to a board, as usual. Notice the messages appearing in the message pane and find the path from where the avrdude runs, along with all avrdude options that are created automatically by the IDE. In my case, it was:

```
/home/myuser/.arduino15/packages/arduino/tools/avrdude/6.3.0-arduino17/bin/avrdude -C/home/thisuser/.arduino15/packages/arduino/tools/avrdude/6.3.0-arduino17
```

where I substituted my home directory with /home/myuser.

There are several other parameters but at this stage we only care about the configuration file, which is provided with option -C.

You can leave the *upload* check-box in the checked state, because we will use the extended messaging in the process.

Locate also the path of file programmers.txt because we will need it below.

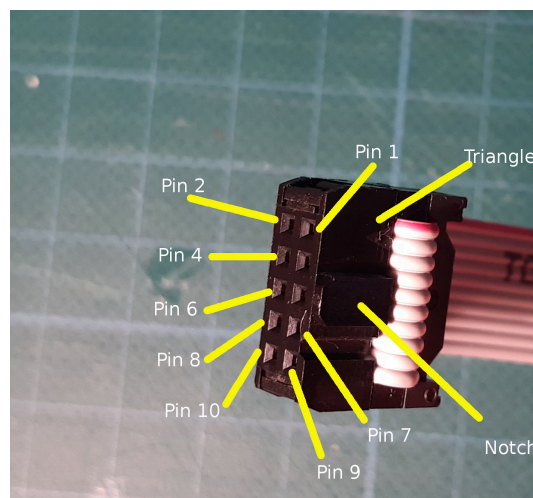
Step 2. Prepare your programmer

1. If you use an Arduino board as programmer, upload the ArduinoISP sketch onto your Arduino board.
2. Wire up the Arduino board with the connector on one side of the ribbon cable.

Arduino - 10-pin connector

Vcc (+5V)	2 Vcc
GND	4 GND
MISO (12)	9 MISO
MOSI (11)	1 MOSI
SCK (13)	7 SCK
(10)	5 RST

Be extra cautious about the numbering of the connector pins. Usually, there will be an arrow or a triangle marker imprinted on the connector that will mark pin number 1. This photo will help:



Note the location of pin 1 in respect to the notch.

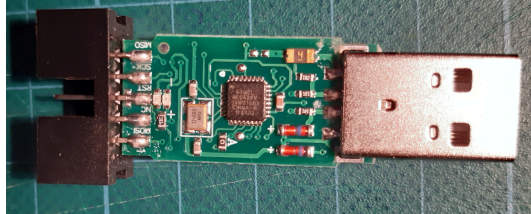
I have also wired a 10uF capacitor between pins RESET and GND of the Nano during my attempts to make things work but this is not necessary.

Step 3. Prepare the USB-ISP

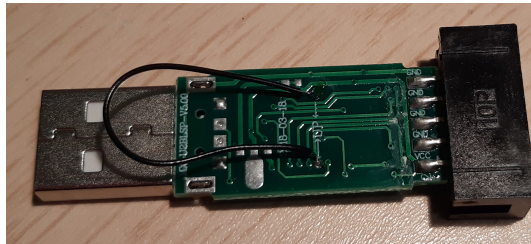
Push the USB connector and the PCB will come out from the box header side.



After taking it out, I noticed that it contains an ATmega88 running at a 12 MHz crystal and the PCB states that it is **MX-U2BLSP-V5.00**, probably mimicking MX-USBISP-V5.00 of another manufacturer. Notice also a 0 Ohm resistor, which is one of the source of our troubles, according to [Making USBasp Chinese Clones Usable by Darell Tan](http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/) [<http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/>].



Using a very thin wire, solder the two pins on the USB-ISP PCB that are marked as "→ UP ←"



Do not connect any cable yet and leave the USB connector unconnected.

Step 4. Re-program USP-ISP with USBasp firmware

Download [usbasp.2011-05-28.tar.gz](https://www.fischl.de/usbasp/usbasp.2011-05-28.tar.gz) [<https://www.fischl.de/usbasp/usbasp.2011-05-28.tar.gz>]. The controller in our USB-ISP is an **ATmega88**, so extract file `/bin/usbasp.atmega88.2011-05-28.hex`.

Rename the file to "usbasp.atmega88.2011-05-28.original.hex".

```
$ mv usbasp.atmega88.2011-05-28.hex usbasp.atmega88.2011-05-28.original.hex
```

Make a copy of the file:

```
$ cp usbasp.atmega88.2011-05-28.original.hex usbasp.atmega88.2011-05-28.hacked.hex
```

Change the file permissions of the original file to read-only, so that we do not modify it by accident.

```
$ chmod 444 usbasp.atmega88.2011-05-28.original.hex
```

Open `usbasp.atmega88.2011-05-28.hacked.hex` with a text editor and make the modification in line 185 of the hex file, as shown in [Darell Tan's blog](http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/) [<http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/>]. Do not bother finding the differences, just change the entire line. The description of this modification is very elegant and is described in detail by [Darell](http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/) [<http://irq5.io/2017/07/25/making-usbasp-chinese-clones-usable/>].

In the ATmega88 HEX file, the corresponding code is located on line 185, and here's the line with the bytes highlighted:

```
:100B8000D9F77A9589F708951BB815B88BEF8AB90C
```

Let's patch the code to set `DDRD = 0b1111_0000`, thus the corresponding new bytes should be `0x80EF` and the correct checksum for that line should now be `0x17`:

```
:100B8000D9F77A9589F708951BB815B880EF8AB917
```

Now it is the time to flash the hacked hex file into the ATmega88. Go to the folder where avrdude is. Use your own path. Mine was:

```
$ cd ~/.arduino15/packages/arduino/tools/avrdude/6.3.0-arduino17/bin
```

Considering that your Arduino board is identified as device `/dev/ttyUSB0`, run:

```
$ ./avrdude -c stk500v1 -b 19200 -P/dev/ttyUSB0 -p atmega328 -vv -C ../etc/avrdude.conf
```

Notice the parameters:

- c stk500v1 -b 19200 : This is to make avrdude communicate with a Arduino ISP, implementing the stk500v1 protocol at 19200bps. We need this setting if our programmer is Arduino (Uno, Nano etc). If you are using another programmer, set the appropriate protocol
- p atmega328 : This is because we still talk to our Arduino programmer (Uno or Nano)
- vv : To enable verbose output
- C ../etc/avrdude.conf : To set the configuration file

We must have an output like this:

```
avrdude: Version 6.3-20190619
        Copyright (c) 2000-2005 Brian Dean, http://www.bdmicro.com/
        Copyright (c) 2007-2014 Joerg Wunsch
```

```
System wide configuration file is "../etc/avrdude.conf"
User configuration file is "/home/myuser/.avrduderc"
User configuration file does not exist or is not a regular file, skipping
```

```
Using Port          : /dev/ttyUSB0
Using Programmer    : stk500v1
Overriding Baud Rate : 19200
AVR Part           : ATmega328
Chip Erase delay    : 9000 us
PAGEL              : PD7
BS2                : PC2
RESET disposition   : dedicated
RETRY pulse        : SCK
serial program mode : yes
parallel program mode : yes
Timeout            : 200
StabDelay          : 100
CmdexeDelay        : 25
SyncLoops          : 32
ByteDelay          : 0
PollIndex          : 3
PollValue          : 0x53
Memory Detail      :
```

Memory	Type	Mode	Delay	Block Size	Poll Indx	Paged	Size	Page Size	#Pages	MinW	MaxW	Polled ReadBack
eeeprom		65	20	4	0	no	1024	4	0	3600	3600	0xff 0xff
flash		65	6	128	0	yes	32768	128	256	4500	4500	0xff 0xff
lfuse		0	0	0	0	no	1	0	0	4500	4500	0x00 0x00
hfuse		0	0	0	0	no	1	0	0	4500	4500	0x00 0x00
efuse		0	0	0	0	no	1	0	0	4500	4500	0x00 0x00
lock		0	0	0	0	no	1	0	0	4500	4500	0x00 0x00
calibration		0	0	0	0	no	1	0	0	0	0	0x00 0x00
signature		0	0	0	0	no	3	0	0	0	0	0x00 0x00

```
Programmer Type : STK500
Description      : Atmel STK500 Version 1.x firmware
Hardware Version: 2
Firmware Version: 1.18
Topcard         : Unknown
Vtarget         : 0.0 V
Varef           : 0.0 V
Oscillator      : Off
SCK period      : 0.1 us
```

avrdude: AVR device initialized and ready to accept instructions

Reading | ##### | 100% 0.02s

```
avrdude: Device signature = 0xff00ff
avrdude: Expected signature for ATmega328 is 1E 95 14
Double check chip, or use -F to override this check.
```

avrdude done. Thank you.

We are at the right track.

Insert the connector at the other end of the ribbon cable to the box header of the USB-ISP. NOTE: LEAVE THE USB CONNECTOR OF THE USB_ISP UNCONNECTED. The board will be powered by our Arduino programmer.

Run the command again.

```
$ ./avrdude -c stk500v1 -b 19200 -P/dev/ttyUSB0 -p atmega328 -vv -C ../etc/avrdude.conf
```

At the end you shall see:

```
avrdude: Device signature = 0x1e930a (probably m88)
avrdude: Expected signature for ATmega328 is 1E 95 14
Double check chip, or use -F to override this check.
```

avrdude done. Thank you.

The “**Device signature = 0x1e930a (probably m88)**“ means that avrdude recognizes the ATmega88 (m88). Do not worry about the “Double check” message. We just wanted to see if we can communicate.

Now it is time to talk to the ATmega88. Run the command, changing the processor definition from -p m328 to -p m88:

```
$ ./avrdude -c stk500v1 -b 19200 -P/dev/ttyUSB0 -p m88 -vv -C ../etc/avrdude.conf
```

At the end, we shall see:

```
avrdude: Device signature = 0x1e930a (probably m88)
avrdude: safemode: lfuse reads as FF
avrdude: safemode: hfuse reads as DD
avrdude: safemode: efuse reads as F9

avrdude: safemode: lfuse reads as FF
avrdude: safemode: hfuse reads as DD
avrdude: safemode: efuse reads as F9
avrdude: safemode: Fuses OK (E:F9, H:DD, L:FF)
```

These are the signature and fuse settings of ATmega88. The USBasp firmware requires those fuses to be: **lfuse FF, hfuse DD**. It seems that the fuses are already programmed to the values that we want. If they were not, we could set them with avrdude, using the following parameters:

```
$ ./avrdude -c stk500v1 -b 19200 -P/dev/ttyUSB0 -p m88 -vv -C ../etc/avrdude.conf -U lfuse:w:0xff:m -U hfuse:w:0xdd:m -U efuse:w:0xf9:m
```

Read the original flash memory of the ATmega88 and save to file. This step is optional but we want first to make sure that we have proper communication and second to keep the current firmware, just in case we would like to restore it for any reason. We shall call this firmware “usbisp_firmware_backup.hex” and store it into the folder you like. Note that we need to provide an absolute path.

```
$ ./avrdude -c stk500v1 -b 19200 -P/dev/ttyUSB0 -p m88 -vv -C ../etc/avrdude.conf -U flash:r:/home/myuser/somefolder/usbisp_firmware_backup.hex:i
```

Make the file read-only, just for protection.

```
$ chmod 444 /home/myuser/somefolder/usbisp_firmware_backup.hex
```

Now, program the USB-ISP with the hacked hex file containing the USB-ASP firmware:

```
$ ./avrdude -c stk500v1 -b 19200 -P/dev/ttyUSB0 -p m88 -vv -C ../etc/avrdude.conf -U flash:w:/home/myuser/somefolder/usbasp.atmega88.2011-05-28.hacked.hex:i
```

The full output is:

```
avrdude: Version 6.3-20190619
Copyright (c) 2000-2005 Brian Dean, http://www.bdmicro.com/
Copyright (c) 2007-2014 Joerg Wunsch

System wide configuration file is "../etc/avrdude.conf"
User configuration file is "/home/myuser/.avrduderc"
User configuration file does not exist or is not a regular file, skipping

Using Port                : /dev/ttyUSB0
Using Programmer           : stk500v1
Overriding Baud Rate      : 19200
AVR Part                  : ATmega88
Chip Erase delay           : 9000 us
PAGEL                     : PD7
BS2                       : PC2
RESET disposition         : dedicated
RETRY pulse               : SCK
serial program mode       : yes
parallel program mode     : yes
Timeout                   : 200
StabDelay                 : 100
CmdexeDelay               : 25
SyncLoops                 : 32
ByteDelay                 : 0
PollIndex                 : 3
PollValue                 : 0x53
Memory Detail              :

                Block Poll
Memory Type Mode Delay Size  Indx Paged  Size   Size #Pages MinW  MaxW     Polled
-----
eeprom      65    20    4    0 no    512    4    0   3600   3600 0xff 0xff
flash       65    6    64    0 yes   8192   64   128  4500   4500 0xff 0xff
lfuse       0    0    0    0 no    1    0    0   4500   4500 0x00 0x00
hfuse       0    0    0    0 no    1    0    0   4500   4500 0x00 0x00
efuse       0    0    0    0 no    1    0    0   4500   4500 0x00 0x00
lock        0    0    0    0 no    1    0    0   4500   4500 0x00 0x00
calibration 0    0    0    0 no    1    0    0    0    0 0x00 0x00
signature   0    0    0    0 no    3    0    0    0    0 0x00 0x00

Programmer Type : STK500
Description    : Atmel STK500 Version 1.x firmware
Hardware Version: 2
Firmware Version: 1.18
Topcard       : Unknown
Vtarget       : 0.0 V
Varef         : 0.0 V
Oscillator    : Off
SCK period    : 0.1 us

avrdude: AVR device initialized and ready to accept instructions

Reading | ##### | 100% 0.02s

avrdude: Device signature = 0x1e930a (probably m88)
avrdude: safemode: lfuse reads as FF
avrdude: safemode: hfuse reads as DD
avrdude: safemode: efuse reads as F9
avrdude: NOTE: "flash" memory has been specified, an erase cycle will be performed
To disable this feature, specify the -D option.
avrdude: erasing chip
avrdude: reading input file "/home/myuser/...../usbasp.atmega88.2011-05-28.hacked.hex"
avrdude: writing flash (4716 bytes):

Writing | ##### | 100% 9.32s

avrdude: 4716 bytes of flash written
avrdude: verifying flash memory against /home/myuser/...../usbasp.atmega88.2011-05-28.hacked.hex:
avrdude: load data flash data from input file /home/myuser/...../usbasp.atmega88.2011-05-28.hacked.hex:
avrdude: input file /home/myuser/...../usbasp.atmega88.2011-05-28.hacked.hex contains 4716 bytes
avrdude: reading on-chip flash data:

Reading | ##### | 100% 4.67s

avrdude: verifying ...
avrdude: 4716 bytes of flash verified

avrdude: safemode: lfuse reads as FF
avrdude: safemode: hfuse reads as DD
avrdude: safemode: efuse reads as F9
avrdude: safemode: Fuses OK (E:F9, H:DD, L:FF)

avrdude done. Thank you.
```

Now, disconnect the IDC connector from USB-ISP.

Also disconnect the Arduino programmer from the USB port. We will not need it any longer.

Unsolder the wire bridge in the – UP – pins of USB-ISP.

It is time to connect USB-ISP to the PC USB port to see what we have done. If all is well, the LED should light BLUE, instead of red before.

See the report of the USB devices

```
$ lsusb
```

```
Bus 003 Device 005: ID 16c0:05dc Van Ooijen Technische Informatica shared ID for use with libusb
```

```
$ dmesg
```

```
[ 3104.019558] usb 3-1.3: new low-speed USB device number 6 using uhci_hcd
[ 3104.155292] usb 3-1.3: New USB device found, idVendor=16c0, idProduct=05dc
[ 3104.155297] usb 3-1.3: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[ 3104.155300] usb 3-1.3: Product: USBasp
[ 3104.155303] usb 3-1.3: Manufacturer: www.fischl.de
```

The device is now recognized as a USBasp with Manufacturer: www.fischl.de [http://www.fischl.de]. This is a sign that flashing the USBasp firmware was successful.

Assemble back the PCB into the metal casing.

We now have a functional USB-ISP programmer running the USBasp firmware. But we have not finished. We must make Arduino IDE able to use this device.

Step 5. Fix Arduino IDE

Start Arduino IDE, select “*Programmer USBasp*” and try to upload Blink example to a bare ATmega328 using *Sketch→ Upload Using Programmer*

You may get an error:

```
avrdude: Warning: cannot open USB device: Permission denied
```

This means that we should instruct Linux to recognize the new USB device and allow users with non-root permission to access it.

In [usbasp.2011-05-28.tar.gz](https://www.fischl.de/usbasp/usbasp.2011-05-28.tar.gz) [https://www.fischl.de/usbasp/usbasp.2011-05-28.tar.gz] there is a folder linux-nonroot . Extract it to the working directory.

The instructions are to copy file 99-USBasp.rules to /etc/udev/rules.d of our Linux PC. But the README file in /etc/udev/rules.d of my Linux Mint, instructs (as per my understanding) to use /lib/udev/rules.d instead.

In addition, it was necessary to make the following two modifications in the contents of file 99-USBasp.rules:

- replace SYSFS with ATTRS
- add GROUP=“dialout” or another group that the user is a member of. Use command `$ groups` to see the current group participation.

Use a text editor to make those changes and finally, 99-USBasp.rules will have the content:

```
# USBasp - USB programmer for Atmel AVR controllers
```

```
SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", ATTRS{idVendor}=="16c0", ATTRS{idProduct}=="05dc", MODE="0666", GROUP="dialout"
```

Now copy it to the destination folder:

```
$ sudo cp 99-USBasp.rules /lib/udev/rules.d/99-USBasp.rules
```

Restart udev, or reboot. Use systemctl if your system runs on systemd

```
$ /etc/init.d/udev restart
```

Next problem: avrdude reports error

```
$ ./avrdude -c usbasp -p m328p -F -C ../etc/avrdude.conf
.....
avrdude: error: program enable: target doesn't answer. 1
```

With the avrdude -F option, I found that the Device signature changes every time I run the command, although it was expected to locate an MCU with 1E 95 0F!

```
avrdude: Device signature = 0xc837d5
avrdude: Expected signature for ATmega328P is 1E 95 0F
```

The problem here is that the chip that we want to program is very slow and cannot respond at the speed of our programmer. Remember that an ATmega328 straight from the factory, without any bootloader, comes preset to 8MHz RC internal clock and CKDIV8 fuse programmed to 0, lowering the clock to 1MHz. To make avrdude speak “slower” we need to specify the parameter -B and set it to a value of 10 or higher.

From http://www.nongnu.org/avrdude/user-manual/avrdude_4.html [http://www.nongnu.org/avrdude/user-manual/avrdude_4.html]

```
-B bitclock
```

Specify the bit clock period for the JTAG interface or the ISP clock (JTAG ICE only). The value is a >floating-point number in microseconds. The default value of the JTAG ICE results in about 1 microsecond bit >clock period, suitable for target MCUs running at 4 MHz clock and above. Unlike certain parameters in the >STK500, the JTAG ICE resets all its parameters to default values when the programming software signs off from >the ICE, so for MCUs running at lower clock speeds, this parameter must be specified on the command-line. It >can also be set in the configuration file by using the 'default_bitclock' keyword.

So the solution is to add -B 10 in avrdude invocations.

Try again

```
$ ./avrdude -c usbasp -p m328p -F -C ../etc/avrdude.conf -B 10
```

and finally a response!

```
avrdude: set SCK frequency to 93750 Hz
avrdude: AVR device initialized and ready to accept instructions

Reading | ##### | 100% 0.01s

avrdude: Device signature = 0x1e950f (probably m328p)

avrdude: safemode: Fuses OK (E:FF, H:D9, L:62)

avrdude done. Thank you.
```

To make the solution permanent, several online articles instruct to modify avrdude.conf and set

```
default_bitclock = 10;
```

But instead of messing with `avrdude.conf` which affects all possible programmers, it is better to edit file `programmers.txt` of your active folder. We have located its path in [Step 1](#).

Find section

```
usbasp.name=USBasp
usbasp.communication=usb
usbasp.protocol=usbasp
usbasp.program.protocol=usbasp
usbasp.program.tool=avrdude
usbasp.program.extra_params=-Pushb
```

and modify the last line to:

```
usbasp.program.extra_params=-Pushb -B 10
```

This modification will force `avrdude` to set parameters `-Pushb -B 10`, in every invocation of USBasp programmer. We will be able to confirm by watching the messages at the IDE message pane. We still have active *File → Preferences Show verbose output during: upload* checked, right?

Finally, I selected *Tools > Programmer: "USBasp"* and uploaded succesfully the Blink program using *Sketch → Upload Using Programmer* to a blank ATmega328.

You now have a working USB-ISP programmer, running USBasp firmware and a fully functional Arduino IDE.

Summary

The original purpose was to purchase a cheap programmer to avoid the clutter of having an Arduino Nano and a bunch of cables, and to avoid wiring manually every time I wished to program an MCU. The result was that I really needed the Nano in order to re-flash the firmware of the USB-ISP and additionally, I have spent a couple of days figuring out how to deal with the situation.

In spite of the fact that now I am the *proud* owner of a USBasp programmer, which along with a ZIF socket will allow me to make my programming process easier (alas only in particular cases because it is not a professional programmer), I have decided to keep my bare-bones Nano programmer with its bunch of wires in a drawer, just in case. As I said at the beginning of this article, this is just for hobbyist fun. Did not expect anything more at a price of less than two euros.

Have fun...

This website uses cookies. By using the website, you agree with storing cookies on your computer. Also you acknowledge that you have read and understand our Privacy Policy. If you do not agree leave the website. [More information about cookies](#)

arduino/how-to-fix-a-usb-isp-programmer-and-make-it-work-with-arduino-ide-on-linux.txt · Last modified: 2024/11/22 12:02 by 127.0.0.1